Semut Adaptive Architecture: A Unified Adaptive Memory Architecture for Intelligent Systems

Timothy Lawrence Sitorus
Semut LLC

July 2, 2026

Abstract

Modern intelligent systems increasingly rely on multiple components, including large language models (LLMs), retrieval systems, workflow engines, knowledge stores, and user feedback mechanisms. While each component has advanced significantly, they are often developed as independent subsystems with limited coordination. Workflow engines optimize execution, retrieval systems manage information access, and feedback mechanisms are frequently confined to isolated evaluation pipelines. As a result, continual adaptation across the entire system remains fragmented. This paper introduces the *Semut Adaptive Architecture*, a unified systems architecture that models intelligent systems through three interconnected memory structures: *Artifact Memory*, *Action Graph*, and *Feedback Memory*. Artifact Memory preserves persistent contextual information such as documents, source code, conversations, and structured knowledge. Action Graph represents executable actions and their dependencies as an evolving graph of operational behavior. Feedback Memory continuously records human evaluations, execution outcomes, environmental observations, and system performance, enabling future decisions to incorporate accumulated experience. Rather than replacing existing reasoning systems, the proposed architecture is reasoning-engine independent. Rule-based systems, large language models, future learning algorithms, and human operators can all utilize the same adaptive memory framework. Retrieved artifacts, historical actions, and prior feedback collectively provide contextual evidence for selecting subsequent actions. The outcomes of execution are subsequently incorporated back into all three memory structures, forming a continuous adaptive loop. The primary contribution of this work is not a new reasoning algorithm, but a unified architectural abstraction that integrates artifacts, actions, and feedback into a coherent adaptive memory system. This abstraction provides a common interface for continual learning, workflow execution, retrieval, and human-in-the-loop collaboration across heterogeneous intelligent systems. Potential applications include personal AI assistants, software engineering agents, enterprise automation, robotics, and multi-agent coordination.

1 Introduction

Recent advances in artificial intelligence have significantly expanded the capabilities of software systems. Large Language Models (LLMs) now enable natural language interaction with documents, software applications, databases, and external services. At the same time, retrieval-augmented generation (RAG), workflow orchestration, persistent memory systems, and agent frameworks have become common architectural components in modern AI applications. Rather than relying on a single model, intelligent systems increasingly consist of multiple interacting subsystems responsible for reasoning, retrieval, execution, and evaluation.

Although these components have individually advanced, they are often designed as largely independent subsystems. Retrieval systems focus on identifying relevant information. Workflow engines manage executable actions and their dependencies. Persistent storage maintains documents and structured knowledge. Human feedback mechanisms evaluate system outputs, while reasoning engines determine subsequent actions. Because these components frequently evolve independently, experience accumulated in one subsystem is not consistently incorporated into the others. As a result, continual adaptation across the entire system remains fragmented.

This fragmentation becomes increasingly important as AI assistants perform longer and more complex tasks. Intelligent systems must not only retrieve information, but also remember previous actions, evaluate outcomes, adapt future behavior, and coordinate interactions across heterogeneous software environments. Existing architectures often require multiple disconnected representations of system state, making it difficult to maintain a coherent history of both human and machine activities.

This paper proposes the *Semut Adaptive Architecture*, a unified systems architecture that organizes adaptive intelligence around three complementary memory structures: *Artifact Memory*, *Action Graph*, and *Feedback Memory*. Artifact Memory preserves persistent contextual information such as documents, conversations, source code, and structured data. Action Graph models executable actions together with their dependencies and observed outcomes. Feedback Memory records evaluations from humans, software systems, and environmental observations that influence future decision making. Rather than treating these memories as isolated data stores, the proposed architecture continuously updates all three through a shared adaptive feedback loop.

An important characteristic of the proposed architecture is its independence from any particular reasoning engine. Rule-based systems, Large Language Models, future learning algorithms, and even human operators can all utilize the same adaptive memory framework. The architecture therefore separates persistent knowledge representation from reasoning itself, allowing reasoning engines to evolve independently while sharing accumulated experience through a common memory interface.

The objective of this work is not to introduce a new reasoning algorithm or memory representation. Instead, it presents an architectural abstraction that

unifies artifacts, executable actions, and accumulated feedback into a coherent adaptive system. By treating these components as interconnected memories rather than isolated subsystems, intelligent systems may better support continual learning, workflow execution, human-in-the-loop collaboration, and long-term adaptation.

The primary contributions of this paper are:

- A unified architectural framework integrating Artifact Memory, Action Graph, and Feedback Memory.
- A reasoning-engine-independent design applicable to rule-based systems, LLMs, and future intelligent agents.
- A continuous adaptive feedback loop that updates all memory structures following execution.
- An architectural perspective for constructing intelligent assistants capable of long-term adaptation across heterogeneous software environments.

The remainder of this paper is organized as follows. Section 2 reviews existing approaches to memory architectures, workflow systems, and AI agents. Section 3 discusses the limitations of fragmented adaptive systems. Section 4 introduces the Semut Adaptive Architecture and its three memory structures. Section **??** describes the adaptive feedback loop connecting reasoning, execution, and memory updates. Finally, the paper discusses implementation considerations, potential applications, limitations, and future research directions.

2 Related Work

Modern intelligent systems have evolved through several complementary research directions, including retrieval systems, workflow orchestration, agent frameworks, persistent memory architectures, and large language models. While these approaches address important aspects of intelligent behavior, they often emphasize different representations of knowledge, execution, and adaptation.

Retrieval-Augmented Generation (RAG) extends language models with external knowledge retrieval, allowing responses to incorporate information beyond the model's training data. Vector databases and semantic search systems provide efficient mechanisms for retrieving documents and structured information. These systems primarily focus on artifact retrieval rather than representing executable actions or accumulating operational feedback.

Workflow orchestration platforms such as Zapier, n8n, Apache Airflow, and similar systems model business processes as directed execution graphs. Their primary objective is reliable task execution, dependency management, and automation across heterogeneous software platforms. Although execution histories and logs are maintained, they are generally treated as operational records rather than adaptive memories that influence future reasoning.

Recent AI agent frameworks, including LangGraph, AutoGen, CrewAI, and related systems, combine reasoning with external tools and workflow execution. These frameworks demonstrate increasingly sophisticated planning capabilities by integrating language models with structured execution. However, persistent memory, workflow representation, and feedback mechanisms are frequently implemented as separate components whose interactions depend on application-specific implementations.

Persistent memory architectures for intelligent assistants have similarly received increasing attention. Conversation histories, vector memories, episodic memories, and knowledge graphs provide mechanisms for preserving contextual information across interactions. Nevertheless, these memories often emphasize information retrieval while leaving action histories and feedback propagation as independent concerns.

Consequently, modern intelligent systems commonly maintain multiple representations of state. Documents and conversations reside in knowledge stores, executable procedures are represented within workflow engines, and evaluations are collected through separate logging or human feedback pipelines. Although each subsystem addresses a specific aspect of intelligent behavior, relatively few architectural frameworks explicitly treat artifacts, actions, and feedback as equally important adaptive memories within a unified representation.

Table 1 summarizes this architectural perspective.

Table 1: High-level comparison of representative intelligent system architectures. The comparison illustrates broad architectural emphasis rather than exhaustive implementation details.

System	Artifacts	Actions	Feedback	Unified Adaptive Memory
RAG Systems	✓	–	Partial	–
Workflow Engines	Partial	✓	Partial	–
Knowledge Graphs	✓	Partial	–	–
AI Agent Frameworks	✓	✓	Partial	Partial
Semut Adaptive Architecture	✓	✓	✓	✓

Unlike existing categories, the Semut Adaptive Architecture does not replace retrieval systems, workflow engines, or reasoning models. Instead, it proposes a common architectural abstraction in which Artifact Memory, Action Graph, and Feedback Memory are treated as interconnected adaptive components. This separation between adaptive memory and reasoning enables multiple reasoning engines, including rule-based systems, large language models, and future learning algorithms, to operate over the same evolving memory structure.

3 Problem Statement

Modern intelligent systems increasingly combine multiple components, including reasoning engines, retrieval systems, workflow orchestration, persistent storage, and human feedback mechanisms. Although each component performs an

important function, they frequently maintain independent representations of system state. As intelligent assistants become capable of performing longer and more complex tasks, maintaining separate representations introduces several architectural challenges.

First, contextual knowledge is often separated from executable behavior. Documents, conversations, and structured information are typically stored within retrieval systems or databases, while executable procedures are represented independently inside workflow engines or application logic. Consequently, reasoning engines must repeatedly reconstruct relationships between retrieved information and available actions.

Second, execution histories are commonly treated as operational logs rather than adaptive memories. Workflow engines record completed tasks and execution status for monitoring and debugging purposes, but these histories are not consistently incorporated into future reasoning. As a result, successful execution patterns and previous failures are difficult to reuse across different tasks.

Third, feedback frequently exists outside the primary decision-making architecture. Human evaluations, software-generated metrics, environmental observations, and execution outcomes are often collected by separate monitoring or evaluation systems. Although these signals provide valuable information about system performance, they rarely participate directly in future retrieval, planning, or execution.

These architectural separations become increasingly significant for long-running intelligent assistants. Systems operating across heterogeneous software environments must continuously coordinate documents, executable actions, user preferences, environmental observations, and accumulated experience. Without a unified adaptive representation, maintaining consistent long-term behavior becomes progressively more difficult as interactions increase.

This work therefore considers the following architectural question:

> How can artifacts, executable actions, and accumulated feedback be represented within a unified adaptive memory architecture while remaining independent of the underlying reasoning engine?

Rather than proposing a new reasoning algorithm, this paper investigates an architectural abstraction that organizes these three complementary forms of memory into a continuous adaptive feedback loop. The following section introduces the proposed Semut Adaptive Architecture and describes the responsibilities of Artifact Memory, Action Graph, and Feedback Memory.

4 Semut Adaptive Architecture

The Semut Adaptive Architecture models intelligent systems as a continuous interaction between reasoning, execution, and adaptive memory. Rather than treating retrieval systems, workflow engines, and feedback mechanisms as independent subsystems, the proposed architecture organizes them around three

complementary memory structures: Artifact Memory, Action Graph, and Feedback Memory.

Figure **??** conceptually illustrates the architecture.

The reasoning engine receives a user request together with context retrieved from the three memory structures. Based on the retrieved information, the reasoning engine determines one or more executable actions. Following execution, observations are recorded as feedback and incorporated back into the adaptive memory. This continuous update process allows future decisions to benefit from accumulated experience.

Unlike architectures that tightly couple memory with a specific reasoning algorithm, the proposed architecture separates adaptive memory from reasoning. Consequently, the reasoning engine may consist of a rule-based system, a Large Language Model, a future learning algorithm, or a human operator. Regardless of the implementation, all reasoning engines interact with the same adaptive memory interface.

The architecture consists of three primary components.

4.1 Artifact Memory

Artifact Memory stores persistent contextual information that may be retrieved during reasoning. Artifacts include both human-generated and machine-generated information such as documents, conversations, source code, structured databases, images, configuration files, and application state.

Unlike conventional storage systems that simply preserve information, Artifact Memory serves as contextual evidence for future reasoning. Retrieval mechanisms may employ semantic search, structured queries, metadata filtering, or other application-specific techniques depending on the underlying implementation.

4.2 Action Graph

The Action Graph represents executable behavior as a directed graph of actions and their relationships. Nodes correspond to executable operations, while edges describe ordering constraints, dependencies, alternative execution paths, or observed transitions.

Rather than representing only predefined workflows, the Action Graph evolves over time as new actions are executed and additional relationships are observed. Historical execution patterns provide valuable context for selecting future actions and coordinating complex tasks across heterogeneous software environments.

4.3 Feedback Memory

Feedback Memory records observations generated during or after execution. Feedback may originate from human evaluations, software systems, environmental sensors, execution metrics, or application-specific performance indicators.

Unlike traditional logging systems, Feedback Memory is intended to influence future reasoning directly. Successful outcomes strengthen confidence in previously executed actions, while failures, corrections, and human preferences become persistent knowledge that guides subsequent decisions.

4.4 Interaction Between Memory Structures

Although each memory structure serves a distinct purpose, they continuously influence one another. Retrieved artifacts provide context for selecting executable actions. Executed actions generate new artifacts and operational history. Feedback evaluates both retrieved information and executed behavior, allowing the adaptive memory to evolve through continual interaction.

The proposed architecture therefore treats artifacts, actions, and feedback as complementary representations of system experience rather than independent software components. This unified representation provides a common adaptive substrate upon which diverse reasoning engines may operate.